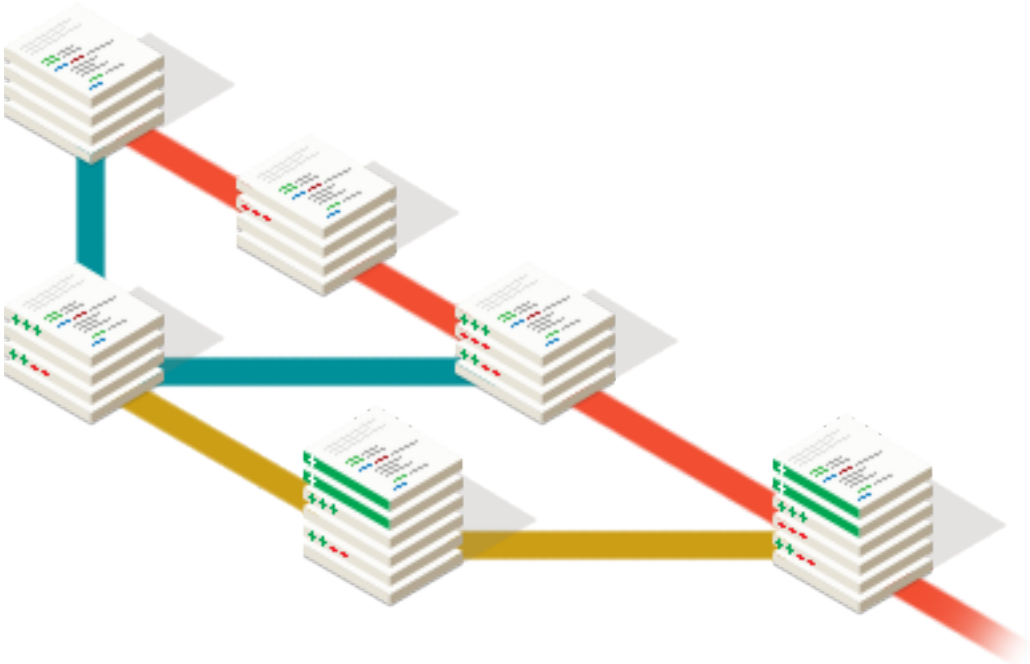


Git: Day 4

MANAGING BRANCHES

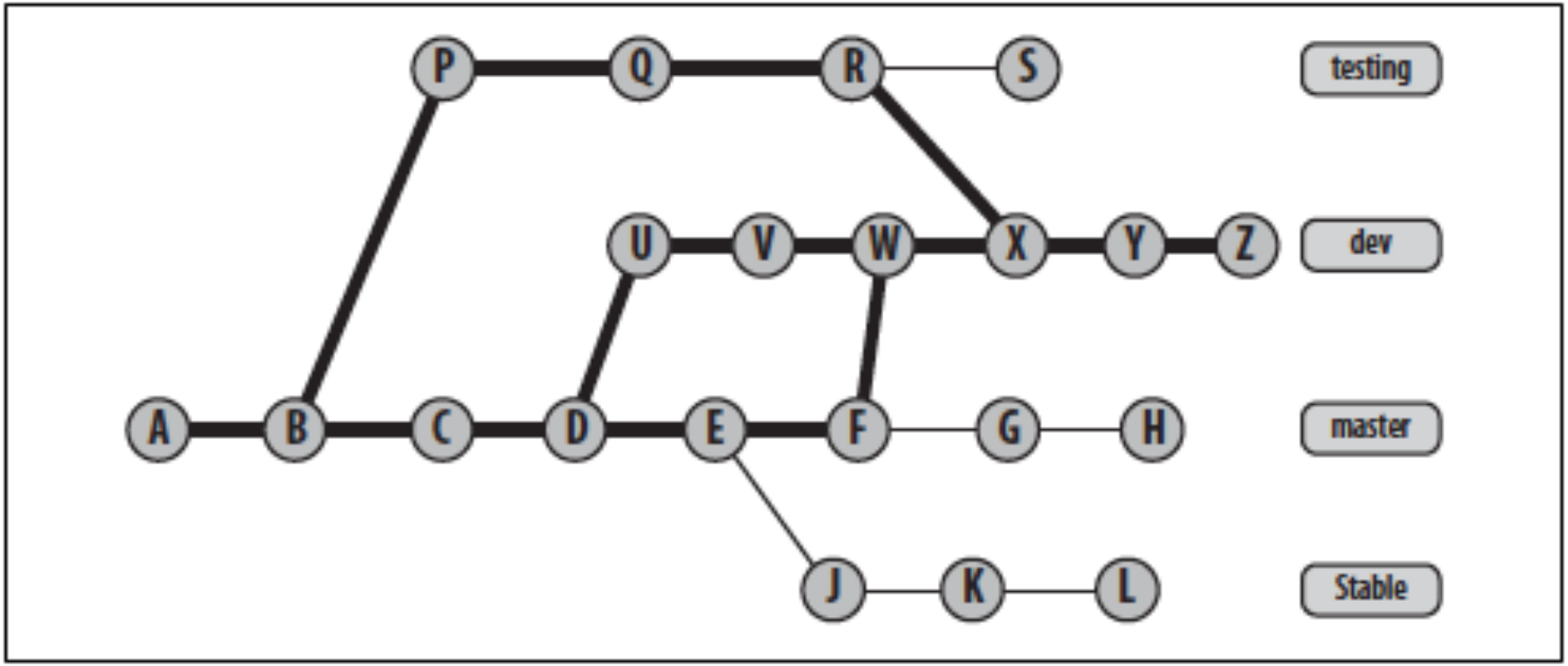
What is a branch

A separated line of development.

A split from a kind of unified, primal state, allowing development to continue in multiple directions **simultaneously** and, potentially, to produce different versions of the project.

Often, a branch is reconciled and merged with other branches to reunite disparate efforts.

What is a branch



Reasons for Using Branches

- An individual customer release (v1.0 – v1.1)
- Encapsulate a development phase (prototype, beta, stable release)
- Isolate the development of a single feature or research into a particularly complex bug.
- Represent the work of an individual contributor.

Branch names

- Default branch in a repo is “master”

- Use an arbitrary name
  - / is allowed (not at the end)
  - No slash-separated component can begin with a dot (**.**). **feature/.new** is invalid
  - Cannot contain two consecutive dots (..)
  - Cannot contain whitespace, **~** , **^**, **:** ,**?**, ***, [**

Using branches

There may be many different branches within a repository at any given time.

There is at most one “active” branch that determines what files are checked out in the working directory.

By default, master is te active branch.

Creating branches

From the active branch:

$ git branch <branch-name>: Only creates a branch

$ git checkout -b <branch-name>: Creates and switches to the new branch

Listing branches

List branches names found in the repository:

$ git branch

```
  bug/pr-1
  dev
* master
```

$ git branch –a

Lists all branches (also tracked branches)

Switching branches

Your working directory can reflect only one branch at a time.

To start working on a different branch:

```
$ git checkout <branch-name>
```

Effects of changing branches:

- Files and directories present in the branch being checked out but not in the current branch are checked out of the object store and placed into your working tree.

- Files and directories present in your current branch but absent in the branch being checked out will be removed from your working tree.

- Files common to both branches are modified to reflect the content present in the checked-out branch.

Delete local branches

You cannot delete the active branch.

```
$ git branch –D <branch>
```

Delete remote branches

2.- Push changes

```
$ git push <remote> :<branch>
```

Pull remote branches

When you clone a repository you can only clone the master branch.

$ git branch –a : View every branch you have got.

$ git checkout <local-branch> <repo>/<remote-branch>: Pull a remote branch into a local branch.

TAGS

Tags vs Branches

A tag is meant to be a static name that does not change or move over time. Once applied, you should leave it alone.

A branch is dynamic and moves with each commit you make. The branch name is designed to follow your continuing development.

Tags vs Branches

You can give a branch and a tag the same name.

If you do, you will have to use their full ref names to distinguish them.

For example, you could use refs/tags/v1.0 and refs/heads/v1.0

Commands

$ git tag : List tags

$ git tag –a <name> -m “text to …”: Create a tag

$ git show <name>

$ git push <repo> --tags: Sends all tags
$ git push <repo> <tag-name>: Sends the tag

Types

- **Lightweight**

Like a branch that doesn’t change. A pointer to a specific commit

- **Annotated**

Are stored as full objects in the Git database. They are checksummed; contain the tagger name, email and date. Can be signed and verified.

STASHING

Description

An intermediate area where you can commit unstable code that you can not deal with.

Use stash when you want to record the current state of the working directory and the index, but want to go back to a clean working directory.

git stashing

A great way to pause what you are currently working on and come back to it later.

$ git stash: Stash your changes away

$ git stash apply: Bring work back

E.g: git stash apply stash@{1}

$ git stash list: List of stashes

git stashing

$ git stash pop: apply the top stash

$ git stash drop <id>: Manually delete stashes

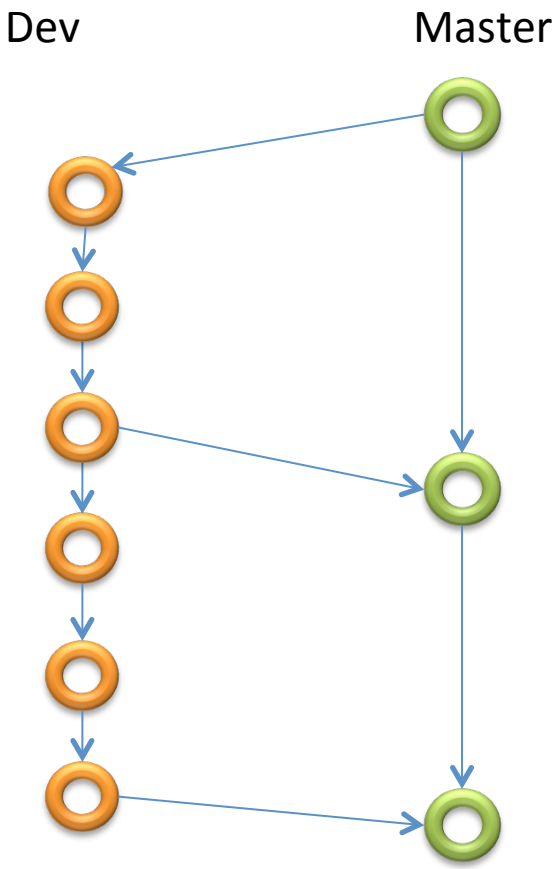
$ git stash clear: Delete al of the stored stashes.

Commands

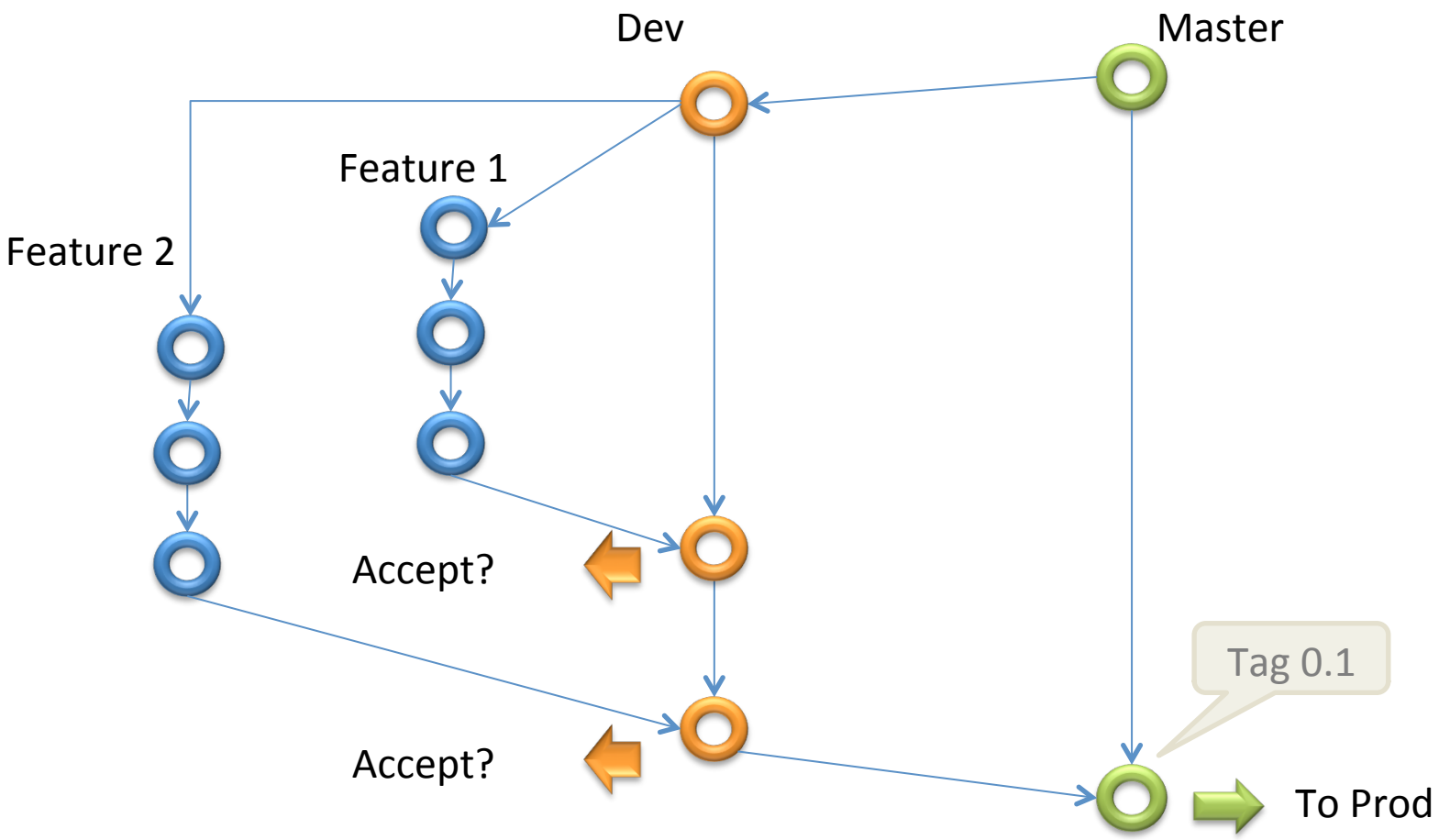
- git stash list
- git stash save "mensaje”
- git stash pop
- git stash show stash@{0}
- git stash apply stash@{0}
- git diff stash@{0}

WORKFLOWS

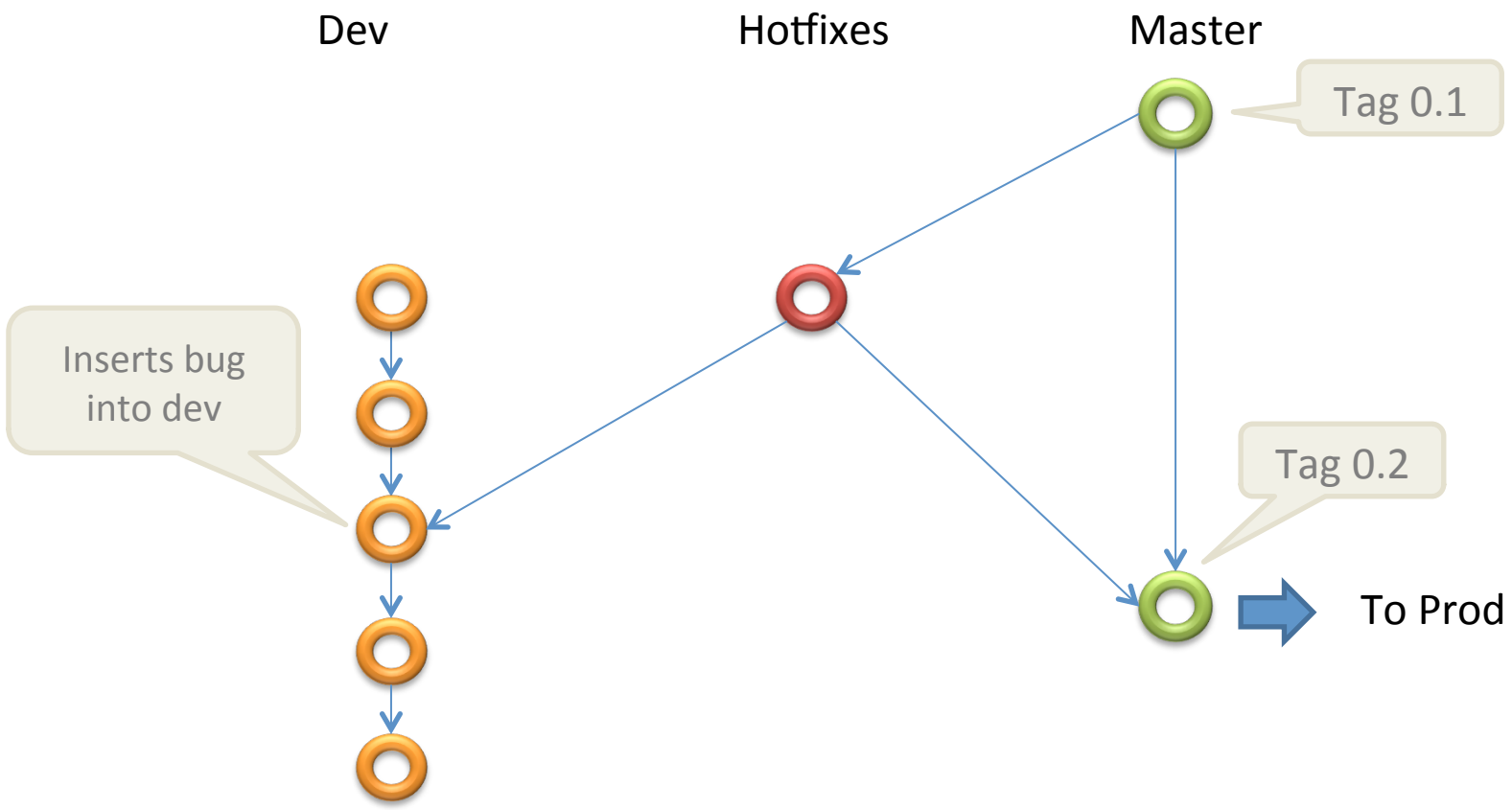
1) Two main branches: Dev & Master



2) One feature / one branch (local)



3) Every bug is fixed in an isolated branch



ALTERING COMMITS: CHERRY PICKS

Altering commits

A commit records the history of your work and keeps your changes sacrosanct, but the commit itself isn't cast in stone.

Git provides several tools and commands specifically designed to help you modify and improve the commit history cataloged within your repository.

Reasons (1)

There are many reasons why you might modify or rework a commit or your overall commit sequence:

- Fix a problem before it becomes a legacy.
- Decompose a large, sweeping change into a number of small, thematic commits.
- Combine individual changes into a larger commit.

Reasons (2)

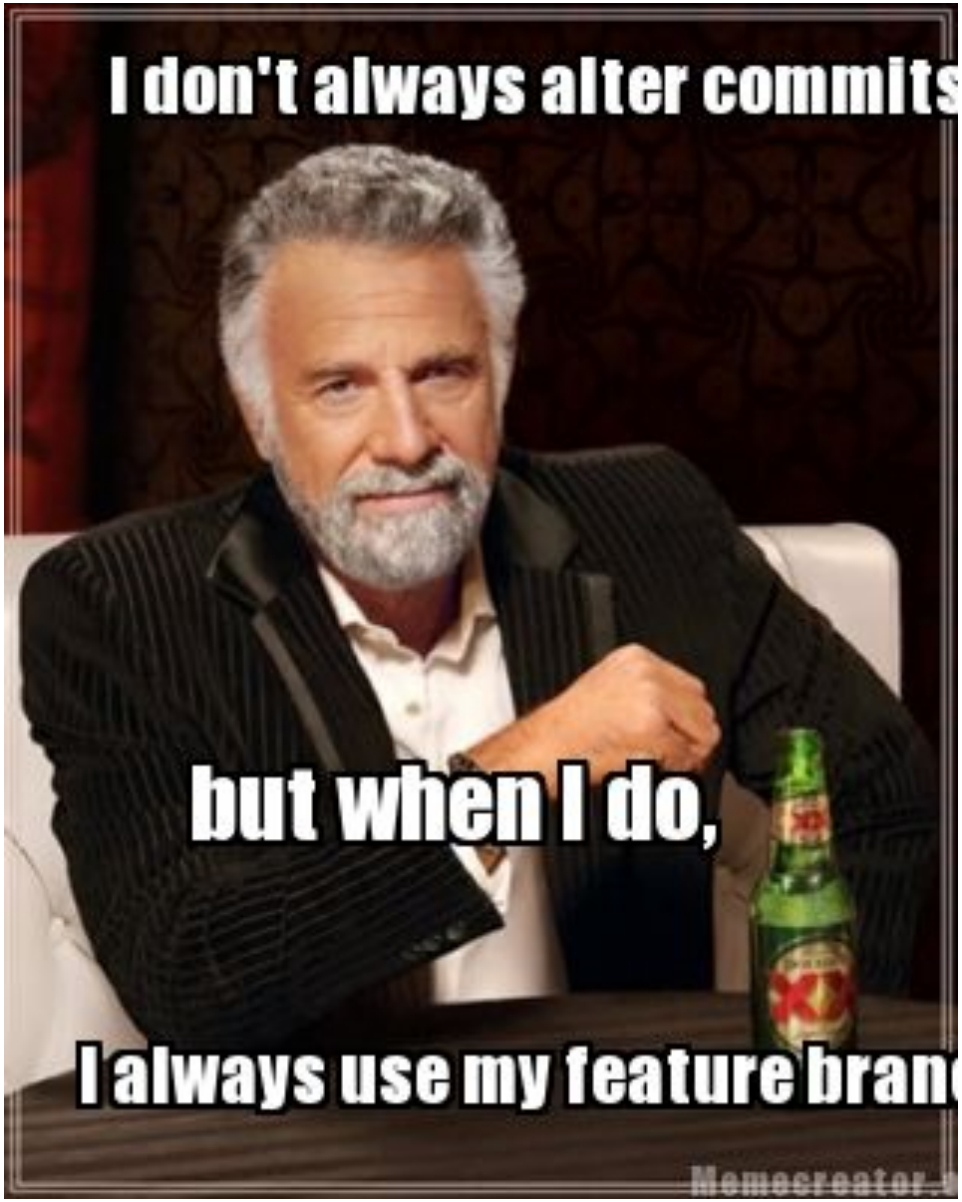
- Incorporate review feedback and suggestions.
- Reorder commits into a sequence that doesn't break a build requirement.
- Order commits into a more logical sequence.
- Remove debug code committed accidentally.

Caution about Altering commits

As a general guideline, you should feel free to alter and improve your repository commit history as long as no other developer has obtained a copy of your repository.

Or, to be more pedantic, you can alter a specific branch of your repository as long as no one has a copy of that branch.

Caution about Altering commits



Please do it into your local branches

Cherry picks

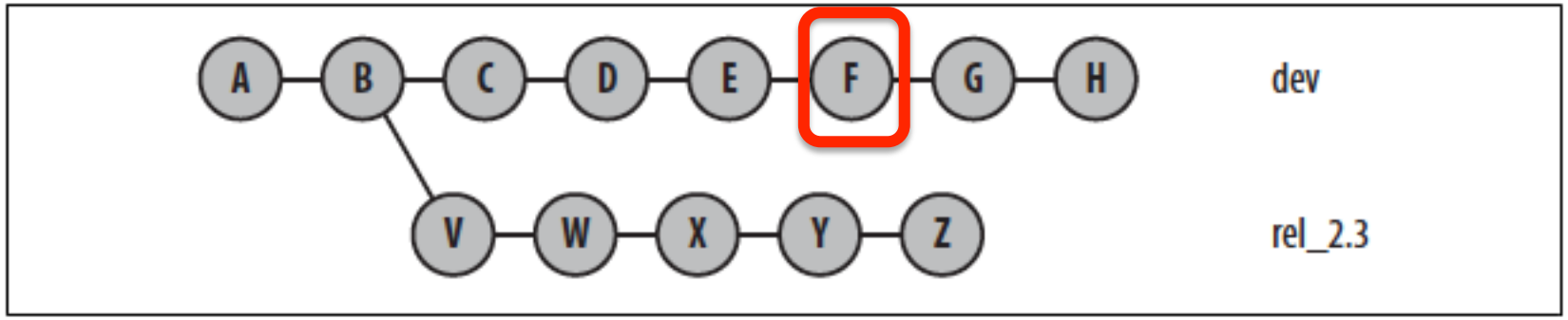
The command git cherry-pick commit applies the changes introduced by the named commit on the current branch.

It will introduce a **new, distinct commit**.

Strictly speaking, using git cherry-pick doesn't alter the existing history within a repository; instead, it adds to the history.

Cherry picks

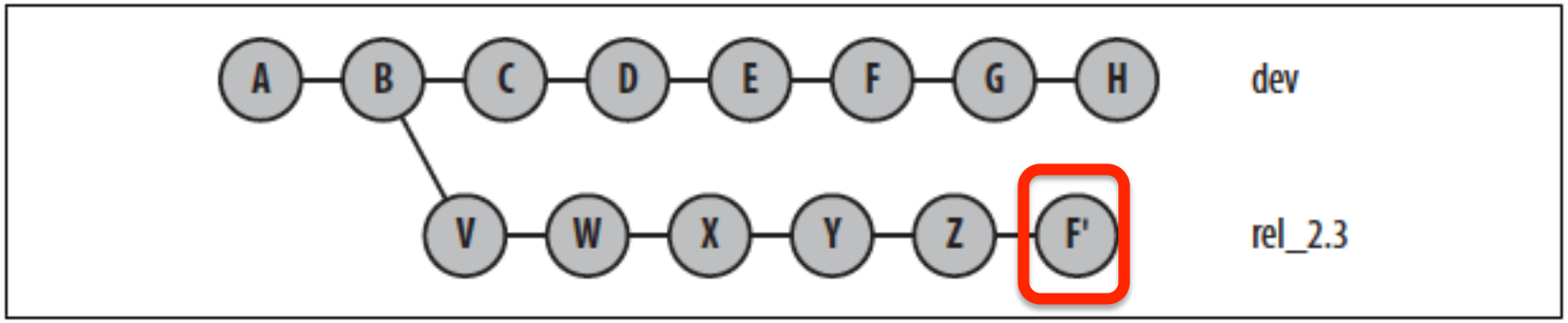
Before



We want to add F to rel_2.3 branch

Cherry picks

After



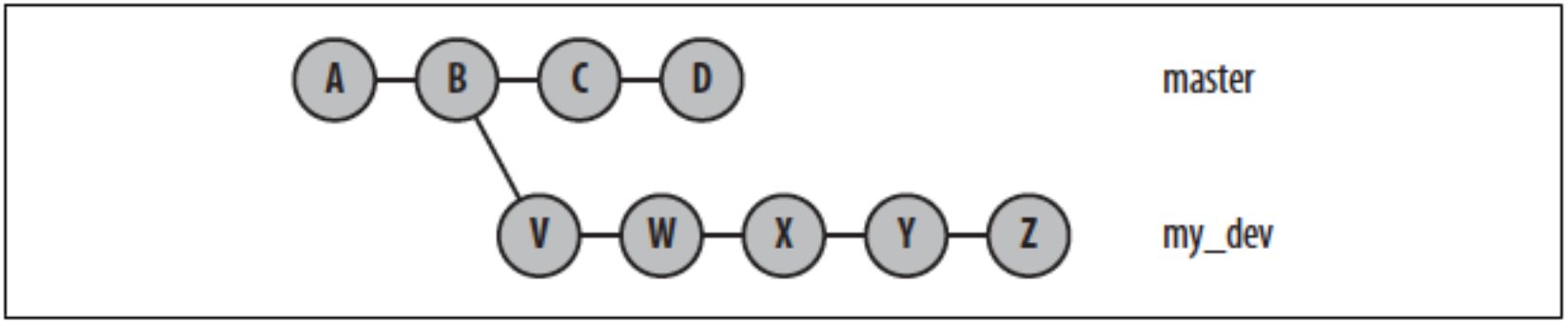

```
$ git checkout rel_2.3
$ git cherry-pick dev~2
```

git Cherry picks

```
$ git checkout master
$ git cherry-pick my_dev^      # Y
$ git cherry-pick my_dev~3     # W
$ git cherry-pick my_dev~2     # X
$ git cherry-pick my_dev       # Z
```

Alter the order

You can alter the order of commits when you apply cherry picks.



New order to master branch Y, W, X, Z

ALTERING COMMITS: REBASES

What is a rebase

The git rebase command is used to alter where a sequence of commits is based.

A rebase helps to cut up commits and slice them into any way and placed exactly where you want them.

This command requires at least the name of the other branch onto which your commits will be relocated.

By default, the commits from the current branch that are not already on the other branch are rebased.

What is a rebase

If a conflict is found, the rebase operation suspends its processing temporarily so you can resolve the conflict and :

rebase --continue: continue to the next commit.

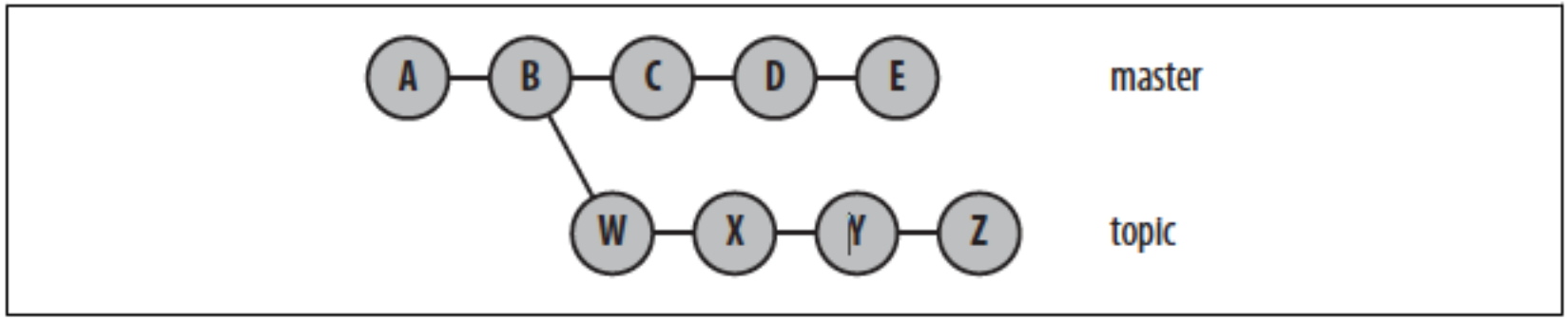
rebase --skip: skip this commit and move to the next.

rebase –abort: abandons the operation and restores the repository to the state prior to issuing the original git rebase.

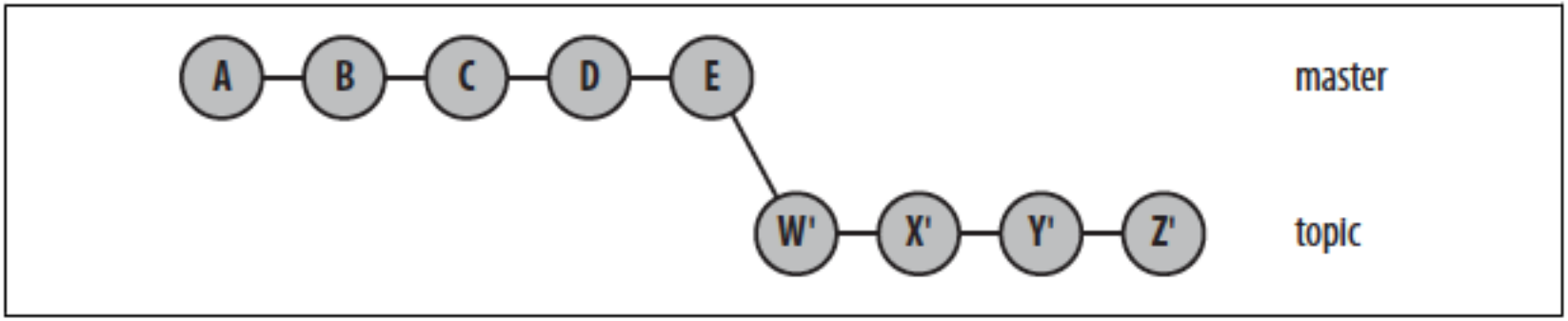
Rebasing commits

Use Case 1: A common use for git rebase is to keep a series of commits that you are developing up-to-date with respect to another branch, usually a master branch or a tracking branch from another repository.

Before git rebase



After git rebase




```
$ git checkout topic
$ git rebase master
```

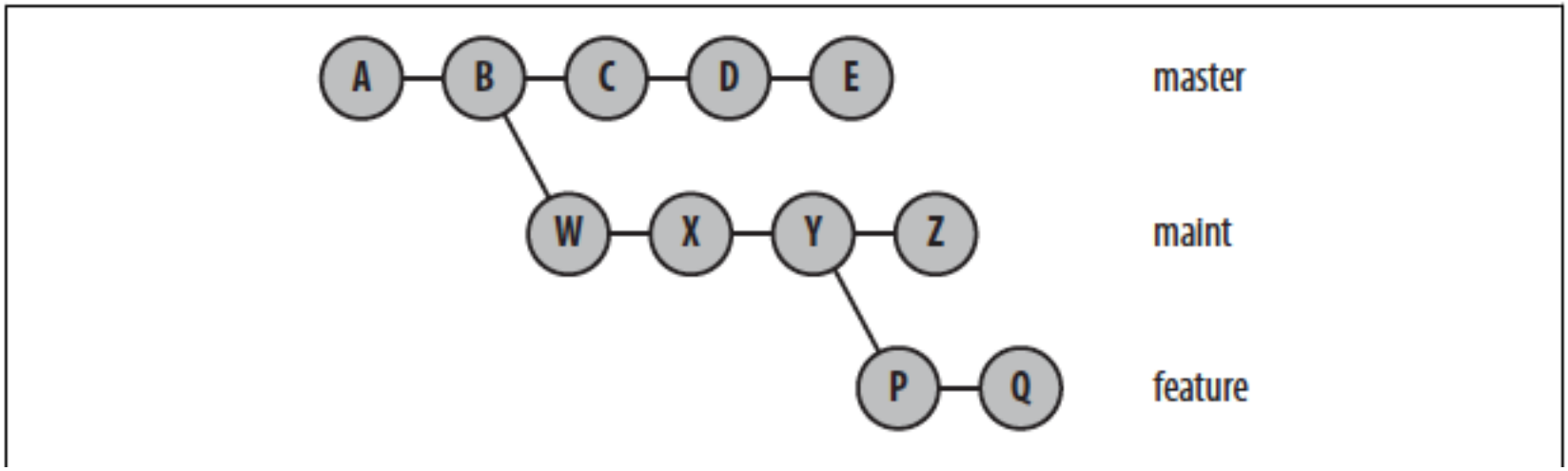
or:

```
$ git rebase master topic
```

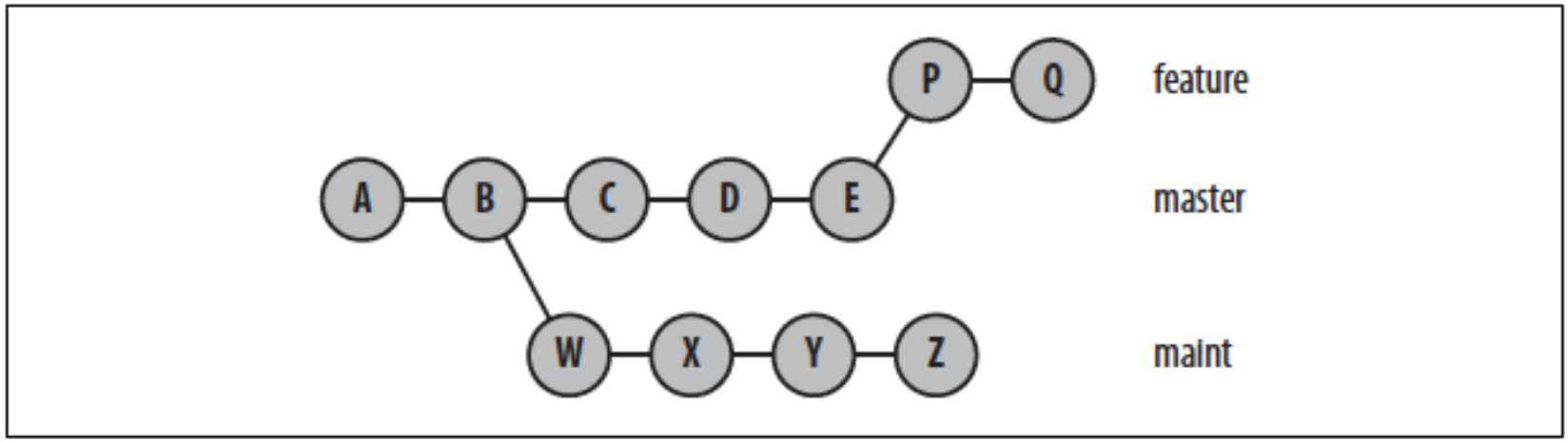
Rebasing commits

Use Case 2: Transplant a line of development from one branch to an entirely different branch using the --onto option.

Before git rebase transplant



After git rebase transplant



git rebase --onto master maint^ feature

Preguntas

Israel Alcázar
israelalcazar@gmail.com
@ialcazar